00 AN IDEA, PUT UP FOR SCRUTINY

GuestGraph.

One explainable guest profile, from data scattered across every system in the hotel.

Robert Blust · Software Engineer & Architect

01 THE PROBLEM

A returning guest looks like five strangers.

Booking.com	PMS	POS	WLAN	Reviews
e.muster@…	MUSTER/ERIKA	Table 12	+41 79 …	„A. M.“

— no shared key —

02 WHY IT IS NOT SOLVED

Matching is easy. Being wrong is expensive.

THE NAIVE FIX

Match on email.

Breaks on shared family addresses, OTA relay addresses, and anyone who changes provider.

WHAT IT COSTS

One guest sees another's stays and invoices.

A wrong merge is a data protection incident — not a cosmetic flaw. So most hotels never try.

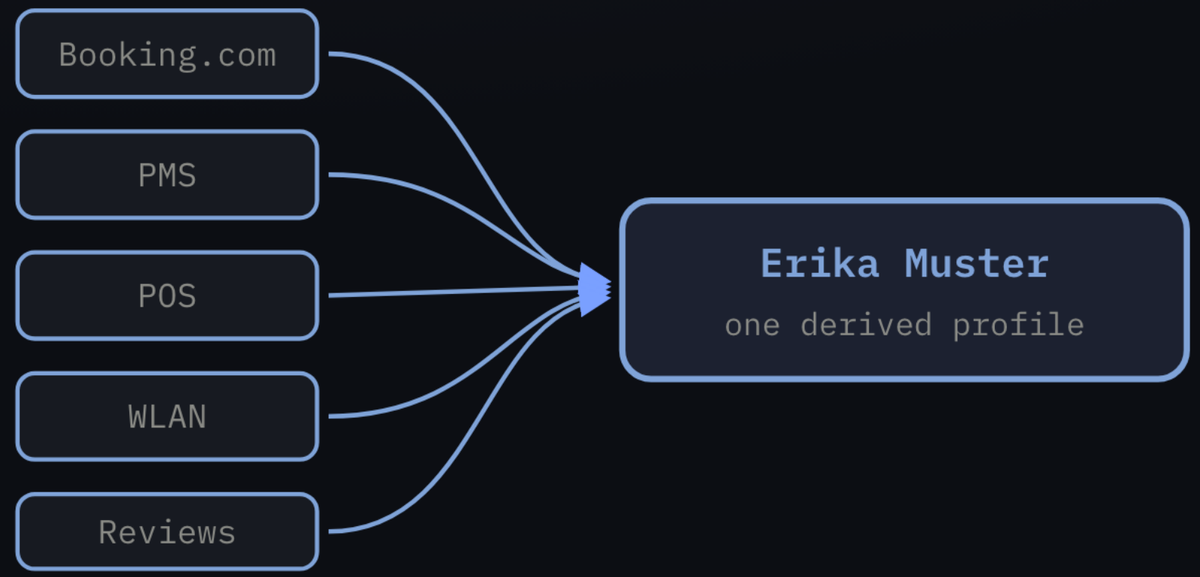
The hard part is not finding matches. It is knowing when not to trust one.

Store what happened. Derive who it was.

Records are immutable
Corrections arrive as new records, never as edits. The original is always what the source actually sent.

The profile is derived
A computation over the records, not a row someone edits. It can always be recomputed.

Tenant-scoped from day one
One instance serves many brands or properties, with no path between them.



Sources stay untouched. **Identity is a conclusion**, not a field.

Not “is it a match?” — but “how sure are we?”

1 · **Shared strong identifier**
Email, phone, loyalty id, ID document → merges outright.

2 · **Probabilistic score**
Merges only above a threshold the operator chose. Otherwise it queues.

3 · **A human decides**
The final word — and their decision sticks against future evidence.

Out of the box, automatic probabilistic merging is **off**. It suggests; a human decides.

05 THE TWO GUARANTEES

Every decision can be explained and undone.

EXPLAIN

"Why are these one guest?"

The full chain: which matcher, how confident, on what evidence, when — and who, if a person or an agent decided it.

UNDO

Split them again — and it holds.

An undo writes a permanent do-not-merge rule, so new evidence cannot silently put them back together.

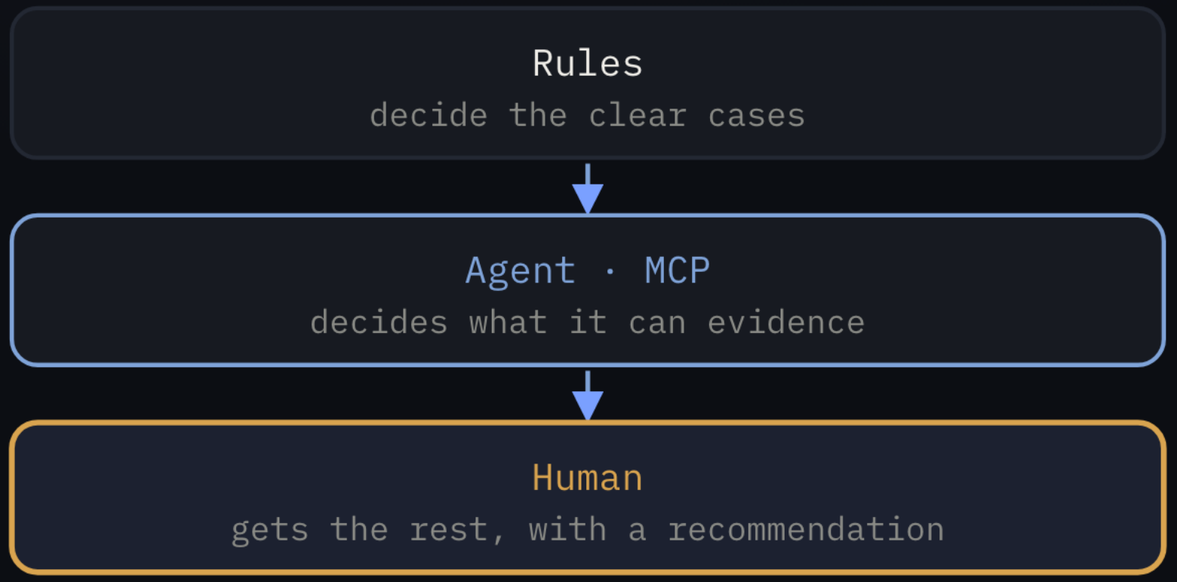
This shipped before any uncertain decision was possible. That order was the point.

An agent is just another uncertain matcher.

Identity comes first
An agent acting on guest data has to know which guest. Without that it answers questions about five strangers.

Gated by machinery that exists
Thresholds, review queue, reversibility — the agent gets no exemption a fuzzy score would not get.

The audit trail names it
Every decision records whether the system, a person, or a named agent made it.



Each tier hands upward what it cannot justify. **Escalation is the default**, not the exception.

Every human decision is a labelled example.

WHAT ALREADY ACCRUES

Feature vector + human verdict.

Every decision stores why it scored what it scored. Every review says confirmed or rejected. On the operation's own data.

WHAT IT ENABLES

Rules today, a model later.

The matcher sits behind one method — candidates in, scored decisions out. A model is its third implementation, not a rebuild.

No model yet, and no pipeline. But the data is accruing in the right shape.

The engine is built. Almost nothing else is.

- ✓ **Identity resolution** — deterministic and probabilistic, with a human review queue
- ✓ **Explain, undo, audit trail** — every decision, including who made it
- ✓ **Guest timeline** — what a guest currently has, not just what was observed
- ○ **Connectors** — real PMS, POS, booking systems. Next, and the part that decides usability.
- ○ **Agent steward, MCP** — designed for, not built. The audit trail already names an agent; nothing acts as one yet.
- ○ **ML-based matching** — no model, no training pipeline. Today the matcher is hand-written rules, and says so in its name.
- ○ **Production use** — none yet. No customers. This is a foundation, not a product.

09 THE MODEL

Open core — because you have to be able to check it.

OPEN SOURCE · APACHE 2.0

Engine, graph, API, connectors.

Run it yourself. Read exactly how it decided to merge two of your guests.

COMMERCIAL (PLANNED)

Hosting, console, MCP for AI agents.

The core never depends on it, and is never degraded to sell it.

A black box that merges guests is not something I would deploy.

10 WHAT I AM ASKING

Tell me where this is wrong.

1 · Is the scattered-guest problem real enough in your operation that someone would pay to fix it?

2 · Would a review queue be used — or ignored, like every other queue?

3 · Which system would have to be connected first for this to be worth anything at all?

4 · Would you let an agent decide a merge — and what would it have to show you first?

github.com/guestgraph · a no is worth more to me than a polite yes